

AI 應用就業養成班

機器學習實作專題手冊

三大實戰專題 × Kaggle 競賽級資料集 × GitHub 作品集

從資料工程、模型迭代、可解釋性到服務部署的完整交付

適用對象	AI 應用就業養成班 受訓學員
專題數量	三選一深做（行有餘力可挑戰第二題）
建議期程	12 週
交付形式	GitHub Repository + 線上 Demo + Demo Day 簡報
文件版本	v1.0

本手冊內容

	章節	重點
PART 1	開始之前	為什麼是這三題、三專題總覽、怎麼選題
PART 2	共同交付標準	Repo 骨架、六項硬性規定、評分量表
PART 3	專題一：信用風險違約預測	表格式機器學習 × 特徵工程 × 可解釋性
PART 4	專題二：電商個人化推薦系統	大規模資料 × 召回排序雙階段 × 線上服務
PART 5	專題三：皮膚病灶多模態辨識	深度學習 × 多模態融合 × 可信度評估
PART 6	資料集下載指南	Kaggle API 設定與各專題下載指令
PART 7	Vibe Coding 開發方法	五步驟工作流、Prompt 範例、三個必答問題
PART 8	12 週課程排程	每週進度與檢核點
附錄 A	三條必須遵守的規則	資料安全與法遵
附錄 B	交付前自我檢核清單	上傳 GitHub 前逐項確認

PART 1 開始之前

1.1 為什麼是這三個專題

先講一個殘酷的事實：Titanic、House Prices、Iris 這類題目，在 GitHub 上有超過十萬份高度雷同的 repo。面試官掃過標題就直接跳過，不會點進去。

本手冊的三個專題全部選自 Kaggle 正式競賽級資料集，它們有三個共同特徵，而這三件事正好是「會跑 scikit-learn」和「能把模型送上線」之間的分水嶺：

- 資料規模大到 **pandas** 會出事 — 你必須真的處理記憶體、真的選對工具。
- 驗證切分一不小心就洩漏 — 你必須真的理解資料的時間性與群組結構。
- 評估指標不是 **accuracy** — 你必須真的把技術指標翻譯成業務決策。

換句話說，這三題無法靠複製貼上完成。也正因為如此，做完之後你手上會有一份真正說得出故事的作品。

1.2 三個專題總覽

	專題主題	對應職缺	難度	建議時數	需要 GPU
P1	信用風險違約預測與可解釋性報告	金融風控 / 資料科學家	★★★★☆☆	40 – 60 小時	不需要
P2	電商個人化推薦系統	推薦系統 / ML Engineer	★★★★★★	60 – 80 小時	選配
P3	皮膚病灶多模態辨識與可信度評估	CV 工程師 / 醫療 AI	★★★★★☆☆	50 – 70 小時	必需

1.3 怎麼選題

三題擇一深做即可，不需要三題都碰。深度遠比廣度值錢——一個做到能上線的專題，勝過三個停在 notebook 的專題。選題建議：

- 想進金融業、或手邊只有筆電沒有 GPU → 選 P1
- 想進電商／平台業、想走 ML Engineer 路線 → 選 P2
- 想做電腦視覺、對醫療 AI 有興趣 → 選 P3

P2 的特別提醒

P2 是三題中最難的一題，難點不在演算法而在資料規模。3,180 萬列交易紀錄用 pandas 做 groupby 會直接 OOM，你必須改用 Polars 或 DuckDB。如果你想在履歷上放「有處理過億級資料的經驗」，這題是最短路徑；但如果你是第一次做完整的 ML 專案，建議先從 P1 開始。

PART 2 共同交付標準

作品集的價值不只在模型分數，更在於「這個 repo 看起來像不像專業工程師寫的」。以下標準三個專題一體適用。

2.1 Repository 骨架

```
project-name/
├── README.md          # 第一屏必須有：問題、成果表、Demo 連結、架構圖
├── MODEL_CARD.md      # 模型用途、限制、已知偏誤、不適用情境
├── Makefile           # make setup / data / features / train / eval / serve
├── pyproject.toml      # 用 uv 管理依賴
├── .gitignore          # 必須排除 data/ 與 models/
├── configs/            # Hydra 或 YAML，禁止硬編碼超參數
├── src/
│   ├── data/           # 下載、驗證、切分
│   ├── features/        # 特徵工程（純函式，可測試）
│   ├── models/          # 訓練與推論
│   └── serving/         # FastAPI
├── tests/              # 至少含 test_no_leakage.py
├── notebooks/          # EDA 專用，不放訓練邏輯
├── reports/figures/    # 所有圖表
└── .github/workflows/ci.yml
```

2.2 六項硬性規定

1. 資料絕對不進 Git。Kaggle 競賽資料多數禁止再散布。改為提供 scripts/download.py，README 寫清楚下載步驟。
2. 隨機種子固定。README 註明「重跑本 repo 應得到 ± 0.002 內的分數」。

3. **CI 必須綠燈。** ruff lint + pytest + 一次用 1% 抽樣資料的 smoke training。GitHub 上那個綠色勾勾就是可信度。
4. **實驗必須追蹤。** 用 MLflow 或 W&B，README 附一張 experiment 對比截圖。
5. **Demo 必須部署。** 部署到 Hugging Face Spaces（免費），連結放 README 最上方。沒有人會 clone 你的 repo 跑起來，但所有人都會點連結。
6. **Notebook 不放訓練邏輯。** notebooks/ 只做 EDA。所有可重複執行的邏輯都要進 src/ 並且可被測試。

2.3 評分量表（100 分）

評分面向	配分	觀察重點
資料處理與驗證設計	20	切分是否正確、有無洩漏、驗證策略是否對應真實部署情境
模型與實驗	25	Baseline → 改進的路徑是否清晰、每一步是否有對照實驗
工程品質	20	可重現性、模組化、測試、CI、配置管理
可解釋性與業務連結	15	有無把技術指標翻譯成業務決策
產品化與 Demo	10	服務可用性、延遲、Demo 完整度
文件與溝通	10	README 敘事、圖表品質、Model Card

不及格條款

出現資料洩漏且未察覺，該項目直接 0 分，不論模型分數多高。

這條規定看起來嚴苛，但它反映的是產業現實：一個洩漏的模型上線後會直接造成商業損失，而且通常在事故發生前沒有人會發現。寧可在課堂上痛一次，也不要在职場上痛一次。

PART 3 專題一：信用風險違約預測與可解釋性報告

一句話定位

從 7 張關聯資料表建構特徵工程流水線，產出一個「能向客戶說明拒貸理由」的風控模型。

3.1 資料集

Kaggle 競賽：Home Credit Default Risk (2018)

網址：<https://www.kaggle.com/competitions/home-credit-default-risk>

檔案	規模 (約)	說明
application_train / test.csv	30.7 萬 / 4.9 萬列， 122 欄	主表，含目標欄位 TARGET
bureau.csv + bureau_balance.csv	170 萬 / 2,700 萬列	聯徵中心其他機構貸款紀錄
previous_application.csv	167 萬列	該客戶在 Home Credit 的歷史申貸
POS_CASH_balance.csv	1,000 萬列	分期／現金貸月頻餘額
installments_payments.csv	1,360 萬列	每期實際繳款紀錄
credit_card_balance.csv	380 萬列	信用卡月頻餘額

總資料量約 2.5 GB | 評估指標 **ROC-AUC** | 正樣本 (違約) 比例約 **8%** | 當年冠軍私榜約 0.805

3.2 五大技術挑戰

挑戰一：關聯表聚合 (本題核心)

2,700 萬列的月頻資料要壓成每位客戶一列。你必須自己設計聚合邏輯：近 3／6／12 個月的逾期次數、額度使用率的變化斜率、繳款金額的變異程度。這是 SQL 與 Polars 的真功夫，不是 `df.describe()`。

挑戰二：記憶體天花板

全部 join 完會直接 OOM。你會被迫學會 dtype downcast、Polars 的 lazy execution、以及分塊處理。這些技巧在職場上每天都用得到。

挑戰三：Target Encoding 的洩漏陷阱

高基數類別特徵用 target encoding 效果很好，但沒做 out-of-fold 就會讓 CV 分數飆高、實際測試崩盤。這是 AI 輔助開發最容易寫錯的地方，也是本題最好的教學素材。

挑戰四：AUC 高不等於能上線

風控要的是校準過的違約機率（PD），不是排序分數。你需要加上 isotonic 或 Platt calibration，並用 Brier score 與 reliability diagram 驗證機率是否可信。

挑戰五：業務指標轉換

把模型分數轉成核准率對壞帳率曲線、KS 統計量、Gini 係數，並算出「在壞帳率不超過 3% 的前提下，最多能核准多少件」的預期損益。這一步做完，你才算真的懂風控。

3.3 里程碑

階段	產出	驗收標準
M0	Repo 骨架 + 資料下載腳本 + CI	make setup && make test 綠燈
M1	只用主表的 LightGBM baseline	CV AUC ≥ 0.75 ，且回報 5-fold 標準差
M2	6 張副表的特徵工程模組	特徵數 700+，make features 可重複執行
M3	模型比較 + 特徵篩選	LGBM / XGB / CatBoost 三方比較表；null importance 篩選前後對照
M4	機率校準 + 業務指標	Reliability diagram、KS、核准率-壞帳率曲線
M5	SHAP 解釋 + 拒貸理由生成	任一申請人可輸出 Top-3 reason codes
M6	FastAPI 服務 + PSI 漂移監控	Docker 起服務，/predict 回傳機率與理由

3.4 讓作品脫穎而出的三件事

- **MODEL_CARD.md** — 寫清楚模型用途、限制、已知偏誤、不該用在什麼情境。這份文件是新人和資深工程師的分水嶺。
- **拒貸理由生成器** — 輸入一筆申請，輸出「您的申請未通過，主要因素：(1) 近 6 個月逾期 2 次 (2) 額度使用率 89% (3) 就職年資不足 1 年」。這在面試 Demo 時殺傷力極強。
- **實驗軌跡表** — README 用一張表列出：baseline 0.752 → +bureau 特徵 0.771 → +行為特徵 0.784 → ensemble 0.791。每一步標註加了什麼、AUC 變化多少、代價是什麼。這證明你在做工程，而不是碰運氣。

3.5 避雷清單

這三個坑幾乎每個人都會踩

AI 很愛直接寫 `df.fillna(df.mean())` 對全表填補——這會把測試集資訊洩漏進來。要求它在 `pipeline` 內用 `fold` 內統計量。

AI 會建議用 SMOTE 過採樣。8% 不算極度不平衡，且 SMOTE 在此題通常掉分。先做對照實驗再決定，不要照抄。

務必寫一個 `tests/test_no_leakage.py`：檢查任何特徵欄位不得由 TARGET 直接推導、fold 之間 SK_ID_CURR 無交集。

想更有挑戰？

進階版競賽：**Home Credit — Credit Risk Model Stability**

<https://www.kaggle.com/competitions/home-credit-credit-risk-model-stability>

它的評估指標把「模型隨時間衰減的幅度」直接算進分數，更貼近真實風控維運場景。

PART 4 專題二：電商個人化推薦系統

一句話定位

處理 3,180 萬筆交易，實作業界標準的召回+排序雙階段架構，並做出可線上查詢的推薦 API。

4.1 資料集

Kaggle 競賽：H&M Personalized Fashion Recommendations (2022)

網址：<https://www.kaggle.com/competitions/h-and-m-personalized-fashion-recommendations>

檔案	規模 (約)	說明
transactions_train.csv	3,180 萬列	2018-09 至 2020-09 兩年完整交易
customers.csv	137 萬列	年齡、會員狀態、電子報訂閱
articles.csv	10.5 萬列	商品分類、顏色、部門、文字描述
images/	10.5 萬張	商品圖，可做視覺 embedding

CSV 約 3.5 GB，含圖片約 35 GB | 評估指標 **MAP@12** | 任務：為每位客戶預測未來 7 天會購買的 12 件商品

4.2 六大技術挑戰

挑戰一：這題 **pandas** 會直接跪

3,180 萬列做 groupby 就崩了。你必須改用 Polars 或 DuckDB——這件事本身就是履歷上的亮點技能。

挑戰二：時間切分是死線

隨機切分會讓「未來」洩漏到訓練集。必須用最後一週當驗證集，再往前滑動做多期驗證。這是推薦系統最常見、也最致命的錯誤。

挑戰三：雙階段架構

單一模型無法從 10.5 萬商品中排序，必須拆成兩層：

- 召回層：多路策略混合——近期熱銷、使用者複購史、同色／同系列共現、item-CF co-visitation matrix、圖片與文字 embedding 的 ANN 檢索。
- 排序層：LightGBM 的 LGBMRanker (lambdarank) 對召回出的候選集做精排。

挑戰四：召回天花板分析

你必須算出「召回集裡到底包含了多少比例的真實答案 (Recall@K)」。如果召回只有 30%，排序做得再好 MAP 也上不去。這個 debug 思維是資深工程師的標記。

挑戰五：負樣本抽樣

正樣本 1 筆對應候選 100 筆，怎麼抽負樣本直接決定排序模型的好壞。這裡沒有標準答案，要靠實驗。

挑戰六：冷啟動

新用戶與新商品沒有歷史行為，必須設計 fallback 策略。真實系統每天都在處理這件事。

4.3 里程碑

階段	產出	验收標準
M0	Repo + Polars 資料管線	3,180 萬列在 8GB RAM 內完成聚合
M1	Popularity baseline	MAP@12 有分數，時間切分驗證框架完成
M2	多路召回	Recall@100 $\geq$ 0.25，並產出各路召回貢獻拆解表
M3	LGBMRanker 精排	MAP@12 相對 baseline 提升 $\geq$ 3 倍

階段	產出	驗收標準
M4	多模態特徵	CLIP 圖片 embedding + 文字 embedding 進特徵，量化增益
M5	FAISS + FastAPI 線上服務	P95 延遲 < 100ms，附壓測報告
M6	Streamlit Demo	輸入 customer_id，顯示 12 張推薦商品圖與推薦理由

4.4 讓作品脫穎而出的三件事

- 視覺化 Demo 是這題的王牌 — 推薦結果直接顯示商品圖，招募方一眼就看懂你做了什麼。務必部署到 Hugging Face Spaces，並把連結放在 README 第一行。
- 召回路徑貢獻拆解圖 — 一張堆疊長條圖顯示每條召回策略各自貢獻多少 Recall，以及加總後的實際效益遞減曲線。
- 延遲對精度的取捨表 — FAISS 用 Flat / IVF / HNSW 三種索引的 recall 與延遲對比。這是 ML Engineer 的核心語言。

4.5 避雷清單

這題的紅線與陷阱

AI 預設會寫 `train_test_split(shuffle=True)`。這題只要出現這行就整份作廢，必須是時間切分。請在你的 SPEC 裡把這條寫成紅線。

MAP@12 的實作有微妙之處（分母是 $\min(\text{真實購買數}, 12)$ ）。要求 AI 寫單元測試，用手算的小例子驗證。

別讓 AI 一開始就跑全量資料。先做 1% 抽樣的 fast path，確認流程正確再放全量。

PART 5 專題三：皮膚病灶多模態辨識與可信度評估

一句話定位

影像加表格資料融合的深度學習模型。重點不在刷分，而在回答一個問題：這個模型能不能被信任？

5.1 資料集

Kaggle 競賽：SIIM-ISIC Melanoma Classification (2020)

網址：<https://www.kaggle.com/competitions/siim-isic-melanoma-classification>

- 訓練影像約 33,126 張，測試約 10,982 張
- 正樣本（惡性黑色素瘤）僅約 584 張，佔 **1.76%**
- 附帶表格 metadata：patient_id、sex、age_approx、anatom_site_general_challenge
- 評估指標 **ROC-AUC** | 需要 GPU（Kaggle Notebook 每週提供約 30 小時免費 T4/P100，足夠完成）

5.2 六大技術挑戰

挑戰一：patient_id 分組是隱形殺手

同一位病人身上多顆痣的影像高度相似。用一般的 StratifiedKFold 會讓同一人的影像同時出現在訓練與驗證集，CV 分數虛高 0.03 以上，然後在測試集崩掉。必須使用 StratifiedGroupKFold。這一點會篩掉八成的人。

挑戰二：1.76% 的極度不平衡

Accuracy 99% 但完全沒用。要用 AUC 與 PR-AUC，並討論在 Sensitivity 95% 這個臨床門檻下，Specificity 還剩多少。

挑戰三：多模態融合

CNN 影像特徵與表格 metadata 要怎麼接？三種方案——late fusion、feature concat 進 MLP head、metadata 當額外 channel——必須做對照實驗，不能憑感覺選。

挑戰四：外部資料整合

併入 ISIC 2019 資料可顯著提升分數，但要處理標註體系差異與重複影像。這是真實專案中最常見的髒活。

挑戰五：不確定性與校準

在醫療場景中，「模型說 0.6 到底是什麼意思」比 AUC 重要。要做 temperature scaling、MC Dropout 或 deep ensemble，並畫出 reliability diagram。

挑戰六：公平性稽核

分別計算不同年齡層、性別、身體部位的 subgroup AUC。如果模型對長者顯著較差，這件事必須寫進報告。這一項是 2026 年 AI 治理的必備素養，而且多數人不會做——做了就是差異化。

5.3 里程碑

階段	產出	驗收標準
M0	Repo + StratifiedGroupKFold 切分	tests/test_split.py 驗證 fold 間 patient_id 零交集
M1	EfficientNet-B0 baseline	CV AUC ≥ 0.85 ，訓練可在 Kaggle 免費 GPU 內完成
M2	資料增強 + 混合精度 + TTA	訓練時間下降、AUC 提升，附 ablation 對照表
M3	影像 + metadata 融合	三種融合方式對照，量化各自增益
M4	多解析度 ensemble + 外部資料	CV AUC ≥ 0.92
M5	校準 + 臨床閾值分析	Reliability diagram、Sens@95% 對應的 Specificity
M6	Grad-CAM + 公平性稽核 + Model Card	Subgroup AUC 表格，明確的 Intended Use / Limitations

5.4 讓作品脫穎而出的三件事

- **Grad-CAM 熱力圖對照** — 真陽性、偽陽性各挑幾張，展示模型看的是病灶還是背景的尺標貼紙。
如果你能找到「模型靠尺標作弊」這種 shortcut learning 案例，那會是面試中極強的故事。
- **公平性稽核表** — 多數人不會做，做了就是差異化。
- **誠實的 Model Card** — 寫出模型不能做什麼，比寫出它能做什麼更能展示專業判斷力。

本專題唯一不可妥協的紅線

必須使用 `patient_id` 做 GroupKFold。AI 產生的程式碼幾乎一定會寫成一般的 StratifiedKFold，請當場改掉，並寫測試確保 fold 之間 `patient_id` 沒有交集。

醫療 AI 的免責聲明（必寫）

MODEL_CARD.md 必須有醒目聲明：本專案僅供學術學習用途，非醫療器材，不得用於任何臨床診斷。

這不只是免責，這展示了你的專業判斷力。任何做醫療 AI 卻沒寫這段的人，在面試時都會被扣分。

PART 6 資料集下載指南

6.1 Kaggle API 設定（所有專題共用）

這一步九成的人會卡住，請務必先做完再開始寫程式。

步驟 1 取得 API Token

前往 <https://www.kaggle.com/settings> → 找到 API 區塊 → 點 Create New Token → 下載 kaggle.json

步驟 2 安裝並放置憑證

```
pip install kaggle
mkdir -p ~/.kaggle
mv ~/Downloads/kaggle.json ~/.kaggle/
chmod 600 ~/.kaggle/kaggle.json
```

步驟 3 到競賽頁面按下同意規則

最常見的失敗原因就在這一步

沒有先在網頁上接受競賽規則，API 下載會直接回 403 Forbidden。

請進入你選定專題的競賽網址，點擊 Join Competition 或 Late Submission，勾選 I Understand and Accept。

步驟 4 驗證安裝成功

```
kaggle competitions list -s "home credit"
```

6.2 專題一 下載指令

Home Credit Default Risk | 約 2.5 GB | 不需要 GPU

```
kaggle competitions download -c home-credit-default-risk -p data/raw
```



```
unzip -q data/raw/home-credit-default-risk.zip -d data/raw
```

解壓後應有以下檔案：

```
application_train.csv  application_test.csv
bureau.csv             bureau_balance.csv
previous_application.csv POS_CASH_balance.csv
installments_payments.csv credit_card_balance.csv
HomeCredit_columns_description.csv
```

6.3 專題二 下載指令

H&M Personalized Fashion Recommendations | CSV 約 3.5 GB，含圖片約 35 GB

建議先只下載 CSV，圖片等做到 M4 多模態階段再下載。

```
kaggle competitions download -c h-and-m-personalized-fashion-recommendations \
-f transactions_train.csv -p data/raw

kaggle competitions download -c h-and-m-personalized-fashion-recommendations \
-f customers.csv -p data/raw

kaggle competitions download -c h-and-m-personalized-fashion-recommendations \
-f articles.csv -p data/raw
```

要做多模態時，再下載完整包（含 10.5 萬張商品圖，約 35 GB）：

```
kaggle competitions download -c h-and-m-personalized-fashion-recommendations -p data/raw
```

工具選擇

這題請直接用 Polars 或 DuckDB 讀取。3,180 萬列用 pandas 做 groupby 會 OOM——這是本專題的第一道關卡，繞不過去。

6.4 專題三 下載指令

SIIM-ISIC Melanoma Classification | 需要 GPU

不要下載官方原始檔

官方提供的是 DICOM 格式，超過 100 GB。請使用社群整理好的 JPEG 版本（下方指令）。

主資料（含 metadata CSV）：

```
kaggle datasets download -d cdeotte/jpeg-melanoma-512x512 -p data/raw
unzip -q data/raw/jpeg-melanoma-512x512.zip -d data/raw/melanoma
```

外部擴充資料（M4 階段使用，可顯著提升分數）：

```
kaggle datasets download -d cdeotte/jpeg-isic2019-512x512 -p data/raw
unzip -q data/raw/jpeg-isic2019-512x512.zip -d data/raw/isic2019
```

解析度選擇策略

同一位作者（cdeotte）另有 256×256、384×384、768×768 等版本，可在其 Kaggle dataset 頁面切換。

建議先用 256×256 把整條 pipeline 跑通，確認 StratifiedGroupKFold 切分正確後，再換到 512×512 正式訓練。這樣可以省下大量 GPU 時數。

6.5 替代題目

如果你想換口味，以下競賽同樣具備競賽級難度，可與講師討論後替換。完整網址為

<https://www.kaggle.com/competitions/> 加上表中的 Slug。

領域	競賽名稱	Slug
表格	IEEE-CIS Fraud Detection	ieee-fraud-detection
時序	M5 Forecasting – Accuracy	m5-forecasting-accuracy
時序	Optiver Realized Volatility Prediction	optiver-realized-volatility-prediction
NLP	Toxic Comment Classification Challenge	jigsaw-toxic-comment-classification-challenge
NLP	Automated Essay Scoring 2.0	learning-agency-lab-automated-essay-scoring-2
CV	Cassava Leaf Disease Classification	cassava-leaf-disease-classification
推薦	OTTO – Multi-Objective Recommender System	otto-recommender-system

PART 7 Vibe Coding 開發方法

先講清楚一件事

AI 輔助開發在機器學習專案有個特殊風險：它會產出跑得動、分數漂亮、但邏輯錯誤的程式碼，而且錯得很安靜。

這不是要你不要用 AI——而是要你用對方法。以下流程就是為了防止這件事。

7.1 五步驟工作流

第一步 先寫 SPEC，不要先寫 code

請 AI 產出 SPEC.md，內容包含：問題定義、資料契約（每張表的 schema 與筆數）、驗證策略、評估指標、紅線清單。你必須逐條審過才進下一步。

我要做 Home Credit Default Risk。先不要寫任何程式碼。

請產出 SPEC.md，包含：資料契約、CV 策略與理由、評估指標，

以及一份「本專案絕對不能做的事」清單（例如任何形式的 target leakage）。

每一個決策都要說明為什麼。

第二步 骨架先行

讓 AI 一次生成完整目錄結構、Makefile、CI 設定、空的模組與測試檔。先讓 make test 綠燈，再開始填內容。

第三步 一次一個里程碑，每個里程碑都要有可執行的驗收指令

不要說「幫我做特徵工程」。要說：

實作 src/features/bureau.py，輸入 bureau.csv 與 bureau_balance.csv，

輸出每個 SK_ID_CURR 一行，包含近 3/6/12 個月的逾期次數與額度使用率。

同時寫 tests/test_bureau.py，用 10 列假資料驗證聚合結果正確。

第四步 主動要求對抗性測試

這一步是本方法論的核心。要主動請 AI 攻擊你自己的程式碼：

請扮演一位嚴格的 code reviewer，審查我的特徵工程流程，
專門找出可能造成 data leakage 的地方。
針對每一個風險，寫一個會失敗的測試來證明它存在。

第五步 你必須能複述

驗收時講師會隨機抽問下列三題。答不出來就是沒學會，退回重做。

#	必答問題	為什麼問這題
1	你為什麼這樣切驗證集？換成隨機切會發生什麼事？	測試你是否理解資料的時間性與群組結構
2	這個特徵是怎麼算出來的？為什麼有用？	測試特徵是你設計的，還是 AI 隨機產生的
3	分數從 0.75 進到 0.79，是哪一步貢獻的？你怎麼確認的？	測試你是否具備實驗設計與歸因能力

記住這件事

面試官問的就是上面這三題，而不是「你用了什麼模型」。模型名稱誰都會背，但只有真的做過的人答得出這三題。

PART 8 12 週課程排程

週次	內容	檢核點
W1 – W2	選題、撰寫 SPEC、建立 Repo 骨架與 CI	講師逐份審 SPEC，未通過不得進入下一階段
W3 – W4	EDA 與 Baseline 模型	期中檢核：驗證策略正確性（本階段最重要）
W5 – W7	特徵工程與模型迭代	每週 15 分鐘 stand-up，報告分數變化與歸因
W8 – W9	可解釋性、機率校準、公平性稽核	產出 reports/figures 下的完整圖表
W10	服務化與 Demo 部署	Hugging Face Spaces 連結可對外存取
W11	README、Model Card、架構圖打磨	同儕互評：請組員在 10 分鐘內看懂你的 repo
W12	Demo Day	15 分鐘簡報 + 技術問答（模擬面試）

附錄 A 三條必須遵守的規則

規則一 資料絕對不可上傳到 GitHub

Kaggle 競賽資料多數禁止再散布。把資料 commit 進去可能違反競賽規則，而且會讓 repo 肥到無法 clone。

專案第一件事就是建立 .gitignore：

```
data/  
models/  
*.csv  
*.zip  
*.parquet  
.kaggle/  
kaggle.json
```

改為在 repo 裡提供 scripts/download.py，README 寫清楚下載步驟。這才是專業做法，面試官看得出差別。

規則二 kaggle.json 是你的帳號憑證

外洩等於帳號被接管。請確認它在 .gitignore 裡，也不要貼進任何 AI 對話框、Slack 訊息或截圖裡。

規則三 先小後大

每個專題都先用 1% 抽樣資料把整條 pipeline 跑通（建議做成 make smoke），確認流程正確再放全量。

不要在最後一天才發現訓練跑不完——這是本課程歷年來最常見的翻車原因。

附錄 B 交付前自我檢核清單

上傳 GitHub 之前，請逐項確認。任何一項打不了勾，就還沒完成。

資料與驗證

- ☐ 驗證切分方式正確（時間切分／GroupKFold），且有測試驗證
- ☐ 已寫 tests/test_no_leakage.py 並通過
- ☐ 所有前處理統計量都在 fold 內計算，沒有用到全量資料
- ☐ data/ 與 models/ 已列入 .gitignore，repo 中不含任何原始資料

模型與實驗

- ☐ 有清楚的 baseline，且 README 記錄了從 baseline 到最終模型的每一步增益
- ☐ 隨機種子已固定，重跑結果可重現
- ☐ 實驗有用 MLflow 或 W&B 追蹤，並附對比截圖
- ☐ 有做過至少一次對照實驗（ablation），而非只調參

工程品質

- ☐ make setup / data / features / train / eval 全部可執行
- ☐ GitHub Actions CI 綠燈（lint + test + smoke training）
- ☐ 超參數放在 configs/，程式碼中沒有硬編碼數字
- ☐ notebooks/ 只有 EDA，訓練邏輯都在 src/

文件與展示

- ☐ README 第一屏有：問題描述、成果表、Demo 連結、架構圖

- ☐ MODEL_CARD.md 完成，包含用途、限制、已知偏誤、不適用情境
- ☐ Demo 已部署且連結可對外存取
- ☐ 所有圖表存在 reports/figures/ 並在 README 中引用

面試準備

- ☐ 能在 3 分鐘內講清楚這個專案解決什麼問題、你做了什麼、結果如何
- ☐ 能回答 PART 7 的三個必答問題
- ☐ 能說出這個模型的三個限制，以及你會怎麼改進

深度遠比廣度值錢。一個做到能上線的專題，勝過三個停在 **notebook** 的專題。